

```
# =====  
# SARI TAKSİ SEGMENTASYON VE KDS SİSTEMİ  
# =====  
  
# 1. AKADEMİK KÜTÜPHANELERİN YÜKLENMESİ  
!pip install ultralytics -q  
  
import cv2  
import numpy as np  
import time  
import pandas as pd  
import matplotlib.pyplot as plt  
from ultralytics import YOLO  
  
# 2. MODEL VE DOSYA AYARLARI  
model = YOLO('yolov8n-seg.pt')  
input_video_path = 'taksi_videosu.mp4'  
output_video_path = 'maskeli_taksiler.mp4'  
  
cap = cv2.VideoCapture(input_video_path)  
  
if not cap.isOpened():  
    print(f"[HATA] '{input_video_path}' bulunamadı! Sol menüden dosya adını kontrol edin.")  
else:  
    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))  
    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))  
    fps = int(cap.get(cv2.CAP_PROP_FPS))  
  
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
```

```
out = cv2.VideoWriter(output_video_path, fourcc, fps, (frame_width,
frame_height))

# Karar Destek Sistemi (KDS) ve Raporlama Parametreleri
threshold_taxis = 3
total_frames = 0
total_taxis_detected = 0
kds_alarm_frames = 0
inference_times = []
channels = 3

sample_interval = 30
sampled_frames = []
sampled_taxi_counts = []

print("-> [SÜREÇ BAŞLADI] Güneş yansımaları eleniyor, sadece gerçek
taksiler maskeleniyor...")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    total_frames += 1
    channels = frame.shape[2]

    # A. RENK ÖN İŞLEME: Güneş parlamalarını elemek için Saturation (130)
ve Value (120) yükseltildi
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lower_yellow = np.array([12, 130, 120])
    upper_yellow = np.array([26, 255, 255])
```

```

yellow_mask = cv2.inRange(hsv, lower_yellow, upper_yellow)

# B. COCO SEGMENTASYON: (conf=0.35, batch=1)
start_time = time.time()
results = model(frame, verbose=False, conf=0.35, batch=1)
end_time = time.time()
inference_times.append((end_time - start_time) * 1000)

combined_mask = np.zeros((frame_height, frame_width), dtype=np.uint8)
frame_taxi_count = 0

for result in results:
    if result.masks is not None:
        clss = result.bboxes.cls.cpu().numpy()
        masks = result.masks.data.cpu().numpy()

        for cls, mask in zip(clss, masks):
            if int(cls) == 2: # Sadece araba (car) sınıfı
                mask_resized = cv2.resize(mask, (frame_width,
frame_height))
                mask_binary = (mask_resized > 0.5).astype(np.uint8) *
255

                # C. ARABANIN KENDİ İÇİNDEKİ SARI ORANINI HESAPLAMA
(Güneş Parlamaşı Engelleme)
                car_pixel_count = np.sum(mask_binary == 255)
                car_local_overlap = cv2.bitwise_and(mask_binary,
yellow_mask)
                yellow_pixel_count = np.sum(car_local_overlap > 0)

                # Arabanın kapladığı alanın yüzde kaçını sarı?

```

```

        yellow_ratio = (yellow_pixel_count / car_pixel_count)
    if car_pixel_count > 0 else 0

    # Sadece gövde alanı en az %8 oranında gerçek sarı
    olan araçları kabul et

    if yellow_ratio >= 0.08:
        frame_taxi_count += 1
        combined_mask = cv2.bitwise_or(combined_mask,
mask_binary)

    total_taxis_detected += frame_taxi_count

    if total_frames % sample_interval == 0 or total_frames == 1:
        sampled_frames.append(total_frames)
        sampled_taxi_counts.append(frame_taxi_count)

# D. KARAR DESTEK SİSTEMİ (KDS)
if frame_taxi_count >= threshold_taxis:
    kds_alarm_frames += 1
    decision = "KRITİK: Yüksek Taksi Yoğunluğu!"
    text_color = (0, 0, 255)
else:
    decision = "NORMAL: Trafik Akisi Stabil"
    text_color = (0, 255, 0)

# E. GÖRSELLEŞTİRME
colored_mask = np.zeros_like(frame)
colored_mask[combined_mask > 0] = [0, 255, 0]
output_frame = cv2.addWeighted(frame, 0.7, colored_mask, 0.3, 0)

cv2.putText(output_frame, f"KDS Durumu: {decision}
({frame_taxi_count})", (30, 40),

```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.8, text_color, 2)

out.write(output_frame)

cap.release()

out.release()

print("-> [VİDEO BİTTİ] Sadece taksiler başarıyla filtrelendi. Rapor
çıktıları hazırlanıyor...\n")

# 3. ADIM: AKADEMİK DOĞRUSAL GRAFİK ÇIKTISI

plt.figure(figsize=(9, 5))

plt.grid(True, which='both', color='gray', linestyle='-', linewidth=0.5)

plt.plot(sampled_frames, sampled_taxi_counts, color='#0055ff',
linestyle='-', linewidth=3, marker='o', markersize=10,
markerfacecolor='#0055ff')

for x, y in zip(sampled_frames, sampled_taxi_counts):
    plt.text(x, y + 0.15, str(y), ha='center', va='bottom', fontsize=11,
fontweight='bold', color='black')

plt.title("Zamana Bağlı Anlık Sari Taksi Yoğunluk Grafiği", fontsize=14,
pad=15)

plt.xlabel("Video Zaman Akisi (Kare / Frame No)", fontsize=12)

plt.ylabel("Tespit Edilen Taksi Sayısı", fontsize=12)

if len(sampled_frames) > 0:
    plt.xlim(min(sampled_frames) - 10, max(sampled_frames) + 10)

if len(sampled_taxi_counts) > 0:
    plt.ylim(0, max(sampled_taxi_counts) + 2)

plt.savefig('taksi_analiz_grafigi.png', dpi=300, bbox_inches='tight')

plt.show()
```

```
# 4. ADIM: TABLO

avg_inference_ms = np.mean(inference_times) if len(inference_times) > 0
else 0

avg_taxis_per_frame = total_taxis_detected / total_frames if total_frames
> 0 else 0

kds_alarm_ratio = (kds_alarm_frames / total_frames) * 100 if total_frames
> 0 else 0

stats_data = {
    "Metrik Bilgisi": [
        "Toplam Islenen Kare Sayisi (Frame Count)",
        "Toplam Tespit Edilen Sari Taksi Sayisi",
        "Kare Basina Ortalama Taksi Yogunlugu",
        "KDS Kritik Yogunluk Alarm Sayisi",
        "KDS Kritik Yogunluk Orani (%)",
        "Ortalama Model Cikarim Suresi (Inference Speed)",
        "Girdi Görüntü Kanal Boyutu (Kanal Sayısı)",
        "Kullanılan Model Batch Size Değeri",
        "Model Algılama Güven Eşiği (Confidence Threshold)"
    ],
    "Koda Göre Doğru Tablo Değeri": [
        f"{total_frames} kare",
        f"{total_taxis_detected} adet",
        f"{avg_taxis_per_frame:.2f} taksi/kare",
        f"{kds_alarm_frames} kare",
        f"%{kds_alarm_ratio:.1f}",
        f"{avg_inference_ms:.2f} ms",
        f"{channels} kanal (BGR)",
        "1 (Batch Size)",
        "0.35 (Confidence Threshold)"
    ]
}
```

```
}  
  
print("\n" + "="*75)  
  
print("MAKALENİN BULGULAR BÖLÜMÜNE EKLENECEK PERFORMANS TABLOSU")  
  
print("="*75)  
  
print(pd.DataFrame(stats_data).to_markdown(index=False))  
  
print("="*75)
```